

Aula 7: Encapsulamento

Programação Orientada a Objetos em Java

Nesta aula, exploraremos um dos pilares fundamentais da POO: o encapsulamento. Aprenderemos como proteger os dados de um objeto, controlar o acesso aos seus atributos e garantir a integridade do sistema.

AULA 7

POO EM
JAVA



Objetivos de Aprendizagem

Ao final desta aula, você será capaz de:

1

Compreender o Conceito

Entender o que é encapsulamento e por que ele é um dos 4 pilares da Programação Orientada a Objetos.

2

Modificadores de Acesso

Aplicar corretamente os modificadores `private`, `protected`, `public` e padrão em Java.

3

Getters e Setters

Implementar métodos de acesso seguindo as convenções de nomenclatura da linguagem Java.

4

Reconhecer Benefícios

Identificar as vantagens do encapsulamento na manutenção, segurança e qualidade do código.



O que é Encapsulamento?

Encapsulamento é o mecanismo de **proteger os dados internos de um objeto**, escondendo os detalhes de implementação e expondo apenas o necessário ao mundo externo. Pense em uma cápsula ou em um cofre: o conteúdo está protegido, e o acesso é controlado.



Proteção de Dados

Restringe o acesso direto aos atributos internos do objeto.



Ocultamento

Esconde os detalhes internos de implementação da classe.



Controle

Define como e quando os dados podem ser acessados ou modificados.



Pilar da POO

Um dos 4 fundamentos da Programação Orientada a Objetos.



Por que Encapsular?

Sem encapsulamento, qualquer parte do código pode alterar os dados de um objeto diretamente, causando comportamentos inesperados e bugs difíceis de rastrear. Encapsular traz diversas vantagens:

Proteger dados

Impede alterações indevidas que possam corromper o estado do objeto.

Manter integridade

Garante que o objeto sempre se encontre em um estado válido e consistente.

Facilitar manutenção

Mudanças internas não afetam o código que usa a classe.

Reduzir acoplamento

Classes tornam-se mais independentes umas das outras.

Modificadores de Acesso em Java

Java oferece quatro níveis de controle de visibilidade. A escolha correta do modificador é essencial para um bom encapsulamento:

private

Acesso **apenas dentro da própria classe**. O nível mais restritivo e o mais indicado para atributos.

protected

Acesso no **mesmo pacote e em subclasses**, mesmo que estejam em pacotes diferentes.

padrão (package)

Sem modificador explícito. Acesso permitido apenas para **classes do mesmo pacote**.

public

Acesso **irrestrito**. Qualquer classe em qualquer pacote pode acessar o membro.

Tabela de Visibilidade dos Modificadores

A tabela abaixo resume onde cada modificador de acesso permite visibilidade. Use-a como referência rápida ao projetar suas classes:

Modificador	Mesma Classe	Mesmo Pacote	Subclasse	Mundo
private	✓	✗	✗	✗
padrão	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

💡 **Regra de ouro:** Sempre comece com o modificador mais restritivo (`private`) e aumente a visibilidade somente quando necessário.

Atributos Privados

Por convenção, **todos os atributos de uma classe devem ser declarados como `private`**. Isso impede que código externo leia ou modifique os valores diretamente, forçando o uso dos métodos de acesso controlados.

Sem Encapsulamento

```
public class Funcionario {  
    public String nome;  
    public int idade;  
    public double salario;  
}  
  
// Qualquer classe pode fazer:  
func.salario = -5000; // PERIGO!
```

Com Encapsulamento

```
public class Funcionario {  
    private String nome;  
    private int idade;  
    private double salario;  
}  
  
// Acesso direto bloqueado:  
// func.salario = -5000; ERRO!
```

Ao tornar os atributos privados, nenhuma classe externa consegue alterá-los diretamente. O acesso controlado passa a ocorrer exclusivamente pelos métodos **getters** e **setters**.

Métodos de Acesso: Getters

Os **getters** são métodos públicos utilizados para **ler** o valor de um atributo privado. A convenção de nomenclatura em Java é usar o prefixo **get** seguido do nome do atributo com inicial maiúscula.

```
public class Aluno {  
    private String nome;  
    private double nota;  
  
    // Getter para o atributo 'nome'  
    public String getNome() {  
        return nome;  
    }  
  
    // Getter para o atributo 'nota'  
    public double getNota() {  
        return nota;  
    }  
}
```

⚠ Para atributos do tipo **boolean**, a convenção é usar o prefixo **is** em vez de **get**. Ex: **isAtivo()**

Aula 7: Encapsulamento - Programação Orientada a Objetos em Java



Métodos de Acesso: Setters

Os **setters** são métodos públicos utilizados para **alterar** o valor de um atributo privado. Além de simplesmente atribuir o valor, eles são o local ideal para **inserir validações** e garantir a integridade do objeto.

Setter sem validação

```
public void setIdade(int idade) {  
    this.idade = idade;  
    // Aceita valores negativos!  
    // Problema de integridade  
}
```

Setter com validação

```
public void setIdade(int idade) {  
    if (idade > 0) {  
        this.idade = idade;  
    } else {  
        System.out.println(  
            "Idade inválida!");  
    }  
}
```

Observe o uso de `this.idade` para diferenciar o atributo da classe do parâmetro do método, que têm o mesmo nome. O `this` sempre referencia o objeto atual.

Validação nos Setters

A validação dentro dos setters é uma das principais razões para encapsular. Ela garante que o objeto **nunca entre em um estado inválido**, independente do que o código externo tente fazer.

Exemplo 1: Idade não negativa

```
public void setIdade(int
idade) {
    if (idade > 0 && idade <
150)
        this.idade = idade;
}
```

Exemplo 2: Nota entre 0 e 10

```
public void setNota(double
nota) {
    if (nota >= 0.0 && nota
<= 10.0)
        this.nota = nota;
}
```

Exemplo 3: Saldo não negativo

```
public void setSaldo(double
saldo) {
    if (saldo >= 0)
        this.saldo = saldo;
}
```

📌 🔑 Regra importante: **toda validação de dados de entrada deve ocorrer no setter**, nunca deixe essa responsabilidade para o código que chama a classe.

Exemplo Completo: Classe Encapsulada

Veja a implementação completa de uma classe `Produto` com todos os conceitos de encapsulamento aplicados: atributos privados, construtor, getters, setters e validações:

```
public class Produto {  
    private String nome;  
    private double preco;  
    private int quantidade;  
  
    // Construtor  
    public Produto(String nome, double preco, int  
quantidade) {  
        this.nome = nome;  
        setPreco(preco); // usa setter com  
validação  
        setQuantidade(quantidade); // usa setter  
com validação  
    }  
}
```

```
// Getters  
public String getNome()      { return nome; }  
public double getPreco()     { return preco; }  
public int getQuantidade()   { return  
quantidade; }  
  
// Setters com validação  
public void setPreco(double preco) {  
    if (preco >= 0) this.preco = preco;  
}  
public void setQuantidade(int quantidade) {  
    if (quantidade >= 0) this.quantidade =  
quantidade;  
}  
}
```


Encapsulamento, Coesão e Acoplamento

O encapsulamento está diretamente ligado a dois princípios essenciais de qualidade de software:

Alta Coesão

Uma classe coesa tem **uma única responsabilidade bem definida**. O encapsulamento favorece a coesão ao manter os dados e os métodos que os manipulam dentro da mesma classe, sem vazar responsabilidades para o exterior.

- Classe faz uma coisa e faz bem
- Atributos e métodos relacionados juntos
- Código mais fácil de entender

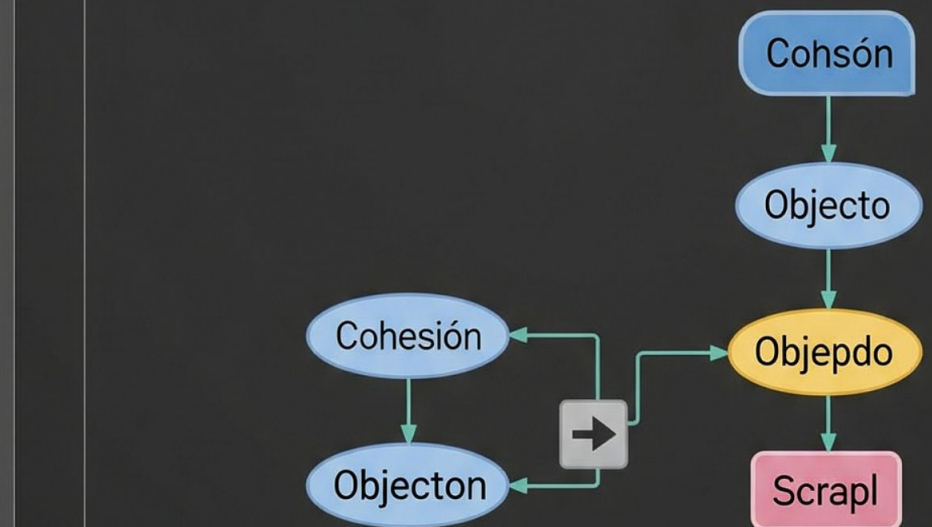
Baixo Acoplamento

Classes com baixo acoplamento são **independentes entre si**. Quando os detalhes internos ficam encapsulados, mudanças em uma classe não forçam mudanças em outras.

- Classes não dependem de detalhes internos umas das outras
- Facilita testes unitários
- Permite evoluir o código com segurança

Aula 7: Encapsulamento - Programação Orientada a Objetos em Java

```
{
    entlin
    if.corin_tond_istne;
    {
        printor.condustopomf() {
            {
                entlxin_atrentertenderdodari(
                    intl_cont..attoatouperstantorhentor();
                    fetuquer();
            };
        }
    }
}
```



```
flet
entd_xh;
r(.shotqun());
}
}
}
```

Erros Comuns ao Encapsular

Conhecer os erros mais frequentes ajuda a evitá-los. Veja as armadilhas que estudantes e desenvolvedores iniciantes costumam cair:



Atributos Públicos

Declarar atributos como `public` sem necessidade destrói o encapsulamento e expõe o estado interno.



Setters sem Validação

Criar getters e setters automaticamente sem nenhuma lógica transforma o encapsulamento em burocracia inútil.



Lógica no Getter

Colocar regras de negócio dentro de getters torna o comportamento da classe imprevisível e difícil de testar.



Expor Coleções Internas

Retornar diretamente uma `List` ou array interno permite que código externo modifique os dados sem controle.

Boas Práticas de Encapsulamento

Seguir boas práticas transforma o encapsulamento de uma técnica em um hábito profissional:

01

Sempre inicie com

`private` todos os atributos como `private` por padrão. Aumente a visibilidade somente quando houver uma justificativa clara.

02

Crie getters apenas quando necessário

Nem todo atributo precisa ser lido externamente. Evite criar getters desnecessários que exponham o estado interno.

03

Valide nos setters

Todo setter deve verificar se o valor recebido é válido antes de atribuir. Esta é a linha de defesa da integridade do objeto.

04

Não exponha coleções internas

Retorne cópias de listas e arrays, ou use `Collections.unmodifiableList()` para proteger estruturas internas.

05

Encapsulamento $\neq$ apenas

~~getters/setters~~ **getters/setters** encapsulamento é sobre controle e responsabilidade. Métodos de negócio devem operar nos dados internamente, sem expô-los.

Exercícios de Fixação

Coloque em prática os conceitos desta aula implementando as seguintes classes em Java:

1

Classe ContaBancaria

Crie uma classe encapsulada com atributos `titular`, `numeroConta` e `saldo`. Implemente métodos `depositar()` e `sacar()` com validação para saldo nunca ficar negativo.

2

Classe Produto

Implemente a classe `Produto` com `nome`, `preco` e `quantidade`. O preço deve ser maior que zero e a quantidade não pode ser negativa. Adicione um método `valorTotalEstoque()`.

3

Classe Aluno

Crie a classe `Aluno` com `nome`, `matricula` e uma lista de `notas`. Implemente setter com validação (nota entre 0 e 10) e um método `calcularMedia()` sem expor a lista interna.



Próxima aula: Herança — como uma classe pode estender outra, reutilizando e especializando comportamentos. O encapsulamento que aprendemos hoje será fundamental para entender como as subclasses interagem com os membros herdados.